Justificativa das decisões de modelagem: Petlife

1. Visão geral

O diagrama de classes representa o sistema Petlife, de gestão administrativa e clínica de uma clínica veterinária, conforme a proposta e o escopo definidos na Etapa 1. Ele está dividido em dois pacotes. O pacote administrativo reúne Cliente, Planos, PlanoBasico, PlanoPremium, Servicos, Faturamento e a interface Faturavel. O pacote clinico reúne Animal, Cachorro, Gato, Consulta, Prontuario, Vacina, RepositorioClinico e as exceções DadosInvalidosException e ElementoNaoEncontradoException. As classes Main e TesteClinico ficaram fora do diagrama por serem o ponto de entrada e os testes, e não entidades do domínio.

2. Identificação das classes e suas responsabilidades

Cada classe guarda apenas os dados e comportamentos que as regras de negócio exigem, o que reflete o princípio da abstração. Cliente representa o tutor e seus dados de contato. Planos e suas especializações definem o desconto e a mensalidade. Servicos forma o catálogo com preço-base. Faturamento calcula o valor da conta aplicando o desconto do plano do cliente. Animal e suas subclasses guardam os dados do paciente. Consulta liga animal e serviço, e ao ser finalizada gera o Prontuario e o Faturamento. Vacina controla aplicação e vencimento. RepositorioClinico concentra o cadastro e a busca, para que as entidades não acumulem essa responsabilidade.

3. Atributos, métodos e visibilidades

Todos os atributos foram definidos como private (-), o que garante o encapsulamento. O acesso é feito por métodos get e set public (+), e

nos set há validações que impedem valores nulos, textos vazios, números negativos e percentuais fora do intervalo de 0 a 100. Na parte clínica, o dado inválido gera a exceção DadosInvalidosException. Os construtores são public para permitir a criação dos objetos. O método gerarFaturamentoAutomatico() da classe Consulta é private, porque é um detalhe interno da finalização da consulta e não deve ser chamado de fora. A visibilidade protected não foi necessária, pois as subclasses acessam os dados da classe-mãe pelos métodos get, preservando o encapsulamento.

4. Herança

Foram identificadas duas hierarquias:

-A primeira é Planos, com as subclasses PlanoBasico e PlanoPremium. Os dois tipos de plano compartilham nome, porcentagem de desconto e mensalidade, que ficam na classe-mãe. Cada subclasse acrescenta o que é exclusivo dela: o PlanoBasico define a quantidade de consultas grátis, e o PlanoPremium define o banho e tosa incluso e o desconto em medicamentos. Planos foi mantida como classe concreta, o que permite representar um plano genérico, e as duas subclasses estendem esse comportamento.

-A segunda é Animal, com as subclasses Cachorro e Gato. Animal é abstrata, pois no sistema não existe um animal genérico, apenas espécies concretas. Ela declara o método abstrato emitirSom(), que cada subclasse implementa de um jeito, o que demonstra o polimorfismo. Cachorro acrescenta raça e raça pura, e Gato acrescenta raça e se costuma morder. Ambas sobrescrevem exibirDadosAnimal() para incluir seus dados próprios. Essa estrutura permite incluir novas espécies sem alterar o restante do sistema.

5. Interface

A interface Faturavel define o contrato de tudo o que pode entrar em uma cobrança, com os métodos calcularValorFinal() e exibirDetalhesFatura(). Ela é implementada por Servicos (valor final igual ao preço-base) e Planos (valor final igual à mensalidade). Uma interface foi escolhida, e não uma herança, porque serviço e plano são conceitos diferentes que não têm relação do tipo “é um”, mas têm em comum a capacidade de serem cobrados. Assim, o sistema fica preparado para tratar de forma uniforme novos itens cobráveis, como procedimentos cirúrgicos ou de estética animal, previstos na Etapa 1.

6. Relacionamentos

Associação Cliente → Planos (0..1): o cliente pode ter um plano ou nenhum, e quando tem, o Faturamento aplica o desconto correspondente.

Associação Animal → Cliente (1): todo animal pertence a um tutor. A associação é unidirecional, o animal conhece o tutor, o que evita dependência circular entre as classes.

Associações do Faturamento (1 Cliente e 1 Servicos): a fatura é calculada a partir do cliente e do serviço prestado.

Associações de Consulta (1 Animal e 1 Servicos) e de Vacina (1 Animal): ligam o atendimento e a vacinação ao paciente.

Composição Consulta ◆ Prontuario e Consulta ◆ Faturamento (0..1): a própria Consulta cria o prontuário (ao ser finalizada) e o faturamento (de forma automática). Esses objetos nascem dependentes da consulta, por isso a relação é de composição.

Agregação RepositorioClinico ◇ Animal, Consulta e Vacina (0..*): o repositório guarda coleções desses objetos (Map para animais, List

para consultas e Set para vacinas), mas eles existem independentemente dele.

Dependências com as exceções: Animal e RepositorioClinico lançam DadosInvalidosException, e RepositorioClinico também lança ElementoNaoEncontradoException. A relação é de dependência, porque as classes apenas usam as exceções, sem guardá-las como atributo.

7. Coerência com a Etapa 1

O diagrama cobre o escopo definido: cadastro e validação de clientes e animais, vínculo de planos de desconto, catálogo de serviços e cálculo de faturamento com abatimento. Também atende aos casos de uso: “Cadastrar cliente”, “Cadastrar animal”, “Vincular plano”, “Emitir faturamento”, “Cadastrar planos” e “Gerenciar serviços” estão no pacote administrativo, e “Registrar consulta” e “Consultar histórico do animal” estão no pacote clínico. Os quatro pilares da orientação a objetos aparecem no modelo: abstração (classes enxutas por domínio), encapsulamento (atributos privados e validação nos set), herança (Planos e Animal) e polimorfismo (emitirSom() e as implementações de Faturavel).